

EECS 598 Final Report

Rui Guo, Naman Saxena
{guorui,namans}@umich.edu

1. Background and motivation

Deep learning and neural networks have become one of the most popular research directions in recent years. Its applications have flooded our daily lives in the form of AI agents, autonomous vehicles, recommendation systems in popular apps, and countless others. In addition to convolutional neural network (CNN) and recurrent neural network (RNN) based models, graph neural network (GNN) has also gained popularity in recent years because of its ability to process unstructured graph data and its wide applications in social network analysis, traffic analysis, molecular analysis. Like any neural network, GNN also requires extensive computation and can be accelerated by parallel computing [1]. However, in contrast to CNNs/RNNs/MLPs, which have structured connectivity and message passing between neurons, GNN is hard to accelerate since a graph, defined by nodes and edges, is also an input to the neural network. Thus GNN poses significant challenges to the acceleration software, requiring an efficient message passing mechanism for unknown and diverse graph structures and an approach to balance loads between different threads in a parallel device.

In this project, we try to optimize the inference speed of a Graph Convolutional Network (GCN) on a GPU, a typical type of GNN. A GCN can be expressed in matrix notation that helps understanding the underlying computation.

Let, $A \rightarrow$ Normalized adjacency matrix of the input graph, usually sparse

$H(l) \rightarrow$ Feature matrix for layer l

$W(l) \rightarrow$ Weight matrix for the vertex combination function

$\sigma(\cdot) \rightarrow$ Non Linear Activation function

Then, the forward propagation from layer l to layer $l + 1$ can be expressed as:

$$H(l + 1) = \sigma(AH(l)W(l))$$

This process includes two operations, *aggregation* $H = AH(l)$ and *combination* $H(l + 1) = \sigma(HW(l))$. The first aggregation operation can be seen as a sparse matrix-matrix multiplication [2] between the row-wise normalized adjacency matrix of the input graph and feature matrix. In contrast, the lateral combination operation is a dense matrix-matrix multiplication [3] with an optional activation function.

In this project, we first implement this GCN computation on a GPU. Then we optimized the two primary operations in GCN computation from views of memory coalescing, shared memory, bank conflict, etc. We also optimize other components such as memory layout in the host device and Cuda streams [4] to improve the performance further. Finally, we do a thorough quantitative analysis and comparison of different optimization tricks. Our optimized program gets at most 61.2x speed up compared with a serial version.

2. Optimizations

This section presents the computation of Graph Convolutional Networks (GCN), which consists of aggregation and combination functions, with our baseline implementation on GPU. We have also optimized the analyzer function from the serial implementation even though it is not a part of the GNN network. Finally, we discuss some other optimizations that further improves GPU's utilization and enhances GCN's performance.

2.1 Baseline GPU implementation

The computation of Graph Convolutional Networks (GCN) mainly consists of two parts, aggregation and combination. In aggregation, features from neighbour nodes are aggregated through a *mean* function. In particular, this process can be formulated as a sparse matrix-matrix multiplication as:

$$H = AH(l)$$

where A is a normalized sparse adjacency matrix with size $N \times N$ for the graph. $A_{ij} > 0$ if there's an edge between node i and node j and the sum of each row in A is normalized to 1. All the positive elements in a same row in A have the same value. $H(l)$ and H are input and output matrices of the aggregation operation respectively, both with size $N \times C_{in}$.

To implement a baseline for this aggregation operation on a GPU, we use the compressed sparse row (CSR) format to represent the graph and calculate the sparse matrix-matrix multiplication, since the representations of graphs in serial version matches with CSR format and need no preprocessing. We launch a grid of $\lceil N/256 \rceil$ 1d blocks with size 256. Each thread is responsible for processing a row representing a node of the output matrix, encompassing all C_{in} features of the node. Each thread will find all corresponding neighbor features associated with that node and calculate the mean of features in all neighbors.

In combination, the aggregated feature matrix H is linearly transformed by multiplication with a weight matrix $W(l)$ and is then optionally passed through a non-linear activation function (for this project, it is ReLU - Rectified Linear Unit). The following equation represents the computation for combination kernel:

$$H(l + 1) = \sigma(HW(l))$$

The output matrix $H(l + 1)$ is then again passed through the next layer of the GNN, going through a similar aggregation and combination process. To implement a baseline for this combination operation on a GPU, we use naive matrix multiplication kernel to multiply the aggregation feature matrix with weight matrix and adjust it to accommodate the output feature biases and the ReLU function. We launch a grid of $\lceil N/16 \rceil \times \lceil C_{out}/16 \rceil$ 2d blocks with a size of 16×16 . Each thread is responsible for processing one of the output matrix element's dot product.

The kernel also adjusts for weight matrix bias for each output feature and passes the outputs through the non-linear activation function.

The analyzer function of the serial implementation, although not part of GNN, does introduce a scope of parallelism. It basically compares the actual output from the two layers of GNN and compares it against reference labels. Each node has a reference label that denotes the maximum feature value for that particular node. As the function traverses across a matrix with dimension $N \times C_{out}$ we can parallelize it. Our baseline implementation involved launching a grid of $\lceil N/512 \rceil$ 1d blocks of 512 length each. Each thread is responsible for finding out the maximum feature value for a particular node, comparing that value against the reference label for that node, performing atomicAdd a value of 1 to a variable, keeping track of all nodes with the correct value. The value of that variable is divided with the total node number for the dataset to get the GNN accuracy.

2.2 Optimization Aggregation

JDS implementation. In addition to the CSR implementation, we also tried Jagged Diagonal Storage (JDS) format with transposed data storage to implement the aggregation step. We first preprocess the graph representation on the host device by sorting nodes according to the number of neighbors and re-laying out the neighbor indices to improve the balance of loads in warps and coalesced memory access in reading neighbor indices. Then we move the preprocessed JDS-transposed graph representation to the GPU and calculate the aggregation step.

2D blocks. In the baseline implementation, we use a single thread to process all C_{in} features for each node, which might assign too many loads on each thread since the feature dimension C is usually large in the first layer. Meanwhile, in small datasets, we may launch too few blocks and threads, which may cause underutilization in the GPU. Therefore, we launch 2D blocks with size 32×16^1 to speed up the process. We launch a 2D grid with $\lceil C_{in}/32 \rceil \times \lceil N/16 \rceil$ blocks. At this time, each thread is responsible for a single element in the output matrix H . We also tried to launch a grid of $\lceil N/32 \rceil$ blocks, in this case, each thread processes $\lceil C_{in}/32 \rceil$ elements.

Transposing features. The serial version code saves features in the host memory in a format of $C_{in} \times N$. If we keep the same format in the GPU, in a row-major system, adjacent columns are features from nodes with adjacent indices (not adjacent in the graph). Memory access to features cannot be coalesced because there's uncertainty in the graph connection. A node in the graph may not be connected to the adjacent nodes in indices. This saving format causes low utilization of memory bandwidth. To solve this problem, we transpose those feature matrices in the memory to a format of $N \times C$. Threads in a warp processes features from the same node, and the memory access is coalesced since they read a continuous 128 Bytes in the global memory.

¹ To further optimize the speed, we use block size of $\min(32, C_{in}) \times 16$ for the aggregation operation. Memory coalescing and thread utilization are both optimized in this way.

We implement the transpose operation as a kernel on the GPU. We also applied optimizations to the transpose kernel. Firstly, we use shared memory to enable coalesced global memory writes of the transposed matrix. Secondly, we add padding in the shared memory to avoid blank conflict while writing from shared memory to global memory.

2.3 Optimization Combination

Tiled Matrix Multiplication. In this optimization, we implement tiled matrix multiplication by dividing the input matrices into Tiles of size $TILE_WIDTH \times TILE_WIDTH$. Every thread is responsible for loading its corresponding input element of both the input matrices from global memory into shared memory, thus reducing global memory reads overlap among various threads significantly and utilizing the much faster-shared memory allocation. Tiling enables reduction in global memory bandwidth by a factor of $TILE_SIZE$ i.e.16 for this project.

Transposed Matrix Multiplication. The serial aggregation function outputs features in the host memory in a format of $C \times N$. If we keep the same format in the GPU, in a row-major system, adjacent columns are features from nodes with adjacent indices (not adjacent in the graph). Therefore, accesses by a thread for a node's input features are not coalesced causing lower memory bandwidth utilization. To fix this, we combine the transposed feature optimization of aggregation to get transposed feature matrices in $N \times C$ format and modify the Naive Matrix Multiplication and Tiled Matrix Multiplication kernels to accommodate the change. Just like the improvement in aggregation, threads in a warp processes features from the same node, and the memory accesses are coalesced.

2.4 Optimization Analyzer

GPU Compare. In the baseline implementation, we use `atomicAdd` to keep track of all nodes with correct maximum feature value, which proves to be costly, resulting in significant performance degradation. In this optimization, our kernel only traverses through the GCN output matrix and compares the maximum feature value for each node with its reference label. The kernel maintains a global output vector and stores a value of 1 for all the nodes that match their reference label. This output vector is then transferred and summed up on the host. This optimization is a significant improvement over the serial and `atomicAdd` version as it also prevents the transfer of the entire feature matrix back to the host after GCN's 2nd combination layer.

Reduction. While the GPU compare optimization is a significant performance improvement, the transfer and sum of the output vector on the host introduces scope for further optimization in the analyzer function. In this optimization, we utilize a reduction kernel (4th kernel taught in class) to perform the sum of GPU compare kernel's output vector, thus parallelizing the sum operation and preventing the output vector's transfer from device to host. In the reduction kernel, each thread loads two input elements into shared memory in a pattern that causes the input element accesses to be sequential, thus eliminating bank conflicts.

2.5 Code flow Optimization

Avoiding data transfer between host and device. We eliminated data transfer from device to host after every aggregation and combination kernel by directly passing the pointer of kernel output to the next stage. This optimization led to a major significant improvement in run times by preventing multiple `cudaMemcpy` schedules.

Streams to overlap transpose kernel with copying graph. After evaluating the code using NVIDIA's visual profiler we realized that the kernel that transposes the input feature matrix to exploit coalesced accesses is independent of `cudaMemcpy` of graph structure to the device. We tried using two streams to overlap the transpose kernel with the `cudaMemcpy` of the graph. However, since the transpose kernel execution time is far less than the copying time and extra overhead is caused by creating streams, the actual running time is slightly longer.

Memory layout in host, and pinned memory. In the serial code version, the dynamic 2D array is allocated by multiple `malloc` calls, thus the memory allocation is not continuous. This causes inefficiency because we have to call `cudaMemcpy` for each row of the matrix (usually 2000 times). After observing the visual profiler output. We find there are a lot of idle intervals between multiple `cudaMemcpy` calls. So it improves the efficiency if we save the data on a continuous memory region and use a single `cudaMemcpy` to copy between host and device. Therefore, we modified the parser code, used a single `malloc` to allocate a contiguous memory region for the whole matrix, and combined multiple calls into a single `cudaMemcpy` call.

In addition to the memory layout, we also used pinned memory to further increase the data transfer speed between host and device.

2.5 Discarded Optimizations

We brainstormed following optimizations but dropped them preliminarily -

Streams for aggregation. We discussed the utility of streams for aggregation, but the uncertainty of the data to be accessed per node due to the graph data structure and the corresponding limited shared memory size makes it rather difficult to use this approach and achieve any significant optimization.

Tiled Matrix Multiplication with Streams. We tried to improve the combination kernel with stream-based tiled matrix multiplication and it yielded significant improvement for the. But when we consider the whole GCN algorithm, eliminating the transfer of feature matrix back to host after aggregation and then copying the data back to the device for combination is an overhead that if avoided will lead to much more performance enhancement as opposed to the improvement with this optimization. As they are mutually exclusive, we dropped this particular optimization.

Combining Aggregation and Combination Kernels. We tried combining aggregation and combination to prevent any data transfer to the host in between these two kernels for one particular layer. This optimization resulted in performance degradation as the logic to synchronize among blocks in the grid just before starting the combination function proved to be extremely costly. Therefore, we dropped this optimization.

3. Evaluation methodology

To evaluate the speed-up of our optimized GCN kernels as opposed to the serial implementation, we captured the individual kernel run times using the NVIDIA profiler (*nvprof* command added to *run_tests*) and complete function run times using C++ *chrono* library functions. We run all our experiments on the Great Lakes cluster, with 1 CPU node and 1 Nvidia Tesla V100 and 32GB RAM. The total run time of the entire code (host + device) is obtained using the time command output from the *run_tests* bash script i.e. CPU execution time (user). NVIDIA's visual profiler has been utilized to evaluate the optimizations' performance enhancements and analyse the scope of further optimizations for the GCN code. For effective speed-up analysis, the run times for all explored optimizations have been captured for the three datasets, i.e., cora, citeseer, and reddit. Further, the data collected has been used to put together the final solution that provides the most optimized GCN GPU implementation across all the functions (aggregation, combination, and analyzer).

4. Speed-up analysis

In this section, we analyze the speed-up of different optimization on several kernels. Execution time measured by NVIDIA's visual profiler are shown and compared in Table 1-6. We compare different combinations of optimizations. In Figure 1, we show the relative speed-up ratio between our optimized kernels and serial version on host. In Figure 2, we draw roofline plots of these optimized kernels, following the guide from GTC 2019 [5][6]. In Table 7, we show the overall run time of our optimized program, averaged over 5 runs.

Optimization					Aggregation kernel execution time					
CSR	JDS ²	2D block	Transpose	One element per thread	cora		citeseer		reddit	
					Layer 1	Layer 2	Layer 1	Layer 2	Layer 1	Layer 2
✓					15.663ms	106.91us	25.253ms	57.696us	12.6281s	1.05665s
	✓				44.041ms	379.17us	72.575ms	210.49us	17.7534s	1.64064s
✓		✓			2.3938ms	16.991us	8.3975ms	13.440us	11.6084s	1.03776s
✓		✓	✓		782.62us	15.136us	1.3517ms	11.936us	537.91ms	31.817ms
✓		✓	✓	✓	115.81us	11.808us	313.82us	8.8960us	483.90ms	28.915ms

Table 1. Aggregation speed-up analysis. We show the kernel execution times of aggregation kernels on a single Tesla V100 with different optimization methods. We show execution times separately for aggregation on two layers in the GCN. Note that transpose will cause an overhead. See Table 3 for the time consumption of the transpose kernel.

² Converting the graph data from CSR format to JDS transposed format needs non-trivial work on the host. Typically $O(n \log(n))$ time complexity for the sorting. On the *reddit* dataset, the time consumption is around 7 seconds.

Optimization			Combination kernel execution time					
Naive	Tiling	Transpose	cora		citeseer		reddit	
			Layer 1	Layer 2	Layer 1	Layer 2	Layer 1	Layer 2
✓			122.75us	4.6400us	323.68us	4.8960us	9.5622ms	648.12us
✓		✓	123.42us	4.7360us	294.56us	4.5760us	8.7140ms	790.17us
	✓	✓	124.64us	4.9280us	318.81us	5.1200us	8.3568ms	674.52us

Table 2. Combination speed-up analysis. We show the kernel execution times of combination kernels on a single Tesla V100 with different optimization methods. We show execution times separately for combination on two layers in the GCN. Note that transpose will cause an overhead, See Table 3 for the time consumption of the transpose kernel.

Optimization			Transpose kernel execution time					
Naive	Shared memory	Remove bank conflict	cora		citeseer		reddit	
			Op 1	Op 2	Op 1	Op 2	Op 1	Op 2
✓			190.21us	3.8400us	588.09us	4.1600us	7.4205ms	428.45us
	✓		76.575us	3.3600us	234.08us	3.3600us	5.2238ms	185.21us
	✓	✓	59.967us	3.0720us	215.77us	3.1040us	5.1533ms	130.46us

Table 3. Transpose speed-up analysis. We show the kernel execution times of transpose kernels on a single Tesla V100 with different optimization methods. There are two transpose operations, transpose the input features and transpose back predictions. We show execution time for these two operations separately.

Optimization			Memcpy execution time					
Naive	Continuous memory	Pinned memory	cora		citeseer		reddit	
			HtoD	DtoH	HtoD	DtoH	HtoD	DtoH
✓			5.0243ms	12.704us	13.944ms	11.744us	139.60ms	2.9397ms
	✓		3.0041ms	7.2640us	9.7287ms	7.5520us	349.46ms	28.899ms
	✓	✓	1.2853ms	6.5280us	4.0194ms	7.5520us	82.861ms	2.9025ms

Table 4. Malloc speed-up analysis. We show the data transfer speed from host to device with several optimization methods. We show the total times for both Host-to-Device and Device-to-Host direction. Note that for naive implementations, there is overhead for calling `cudaMemcpy` which is not included in the table, which may result in a total of around 300ms overhead of around 1300 calls on the reddit dataset.

Optimization			Analyzer execution time					
Naive CPU	GPU compare	Reduction	cora		citeseer		reddit	
			Kernel	Total	Kernel	Total	Kernel	Total
✓			-	191us	-	198us	-	89.218ms
	✓		4.6400us	74us	4.5440us	74us	292.99us	1.351ms
	✓	✓	9.216us	77us	9.472us	80us	302.43us	561us

Table 5. Analyzer speed-up analysis. We show the execution time of the analyzer function with several GPU optimizations. Kernel execution time and total time are shown in the table. Note that the last row, kernel execution time includes the compare kernel and the reduction kernel.

Optimization			Analyzer execution time					
Naive CPU	GPU compare	Reduction	cora		citeseer		reddit	
			Kernel	Total	Kernel	Total	Kernel	Total
✓			-	191us	-	198us	-	89.218ms
	✓		4.6400us	74us	4.5440us	74us	292.99us	1.351ms
	✓	✓	9.216us	77us	9.472us	80us	302.43us	561us

Table 6. Total Run Times speed-up analysis. We show the execution time of the analyzer function with several GPU optimizations. Kernel execution time and total time are shown in the table. Note that the last row, kernel execution time includes the compare kernel and the reduction kernel.

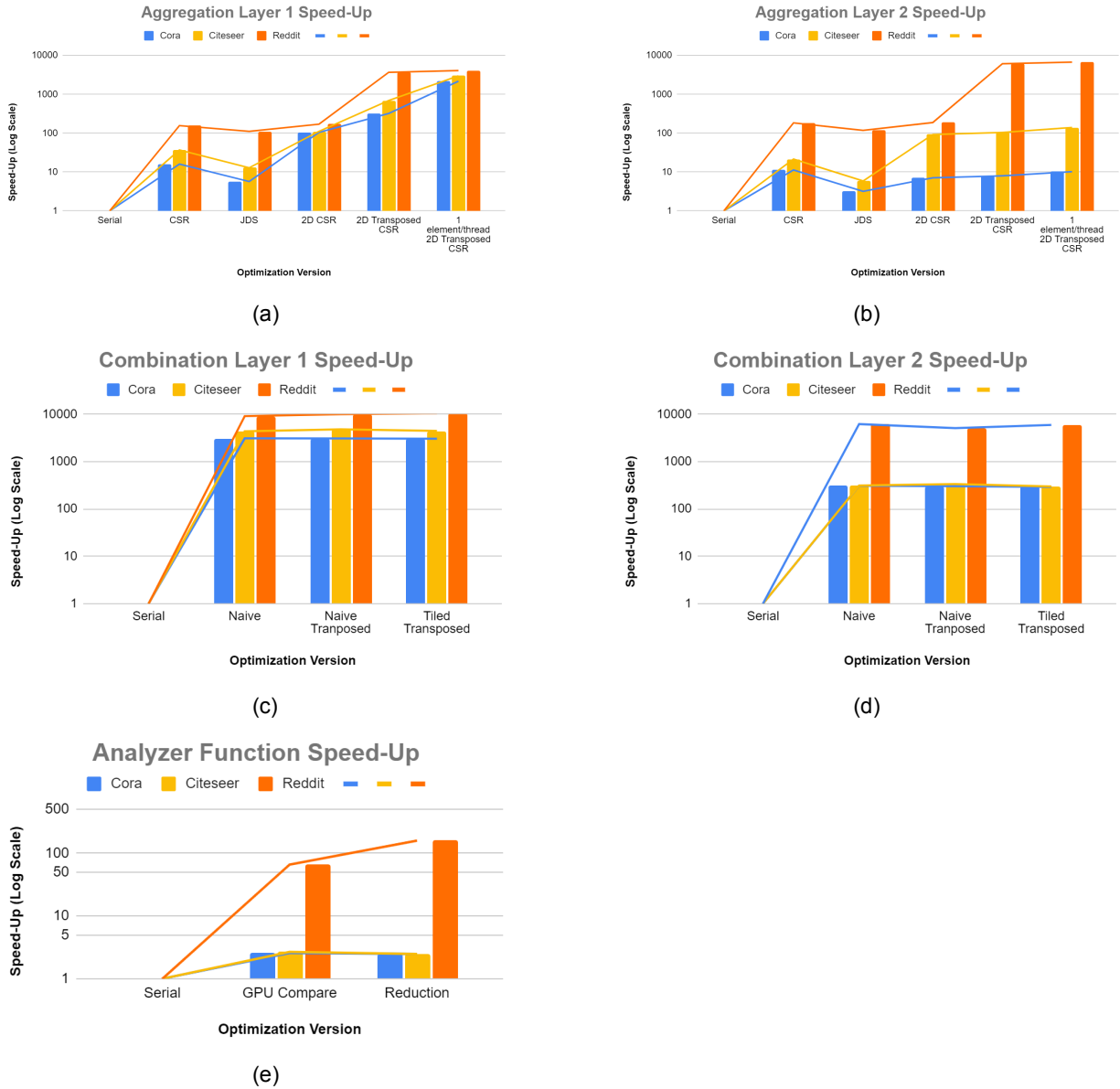


Figure 1. Relative speed up plots of our optimized version versus the serial version for all provided datasets. We show the speedups of individual functions relevant to the GCN code with every kernel specific optimization. [a,b,c,d] reflect aggregation and combination kernel's speedup; [e] reflects speedup of the entire analyzer function.

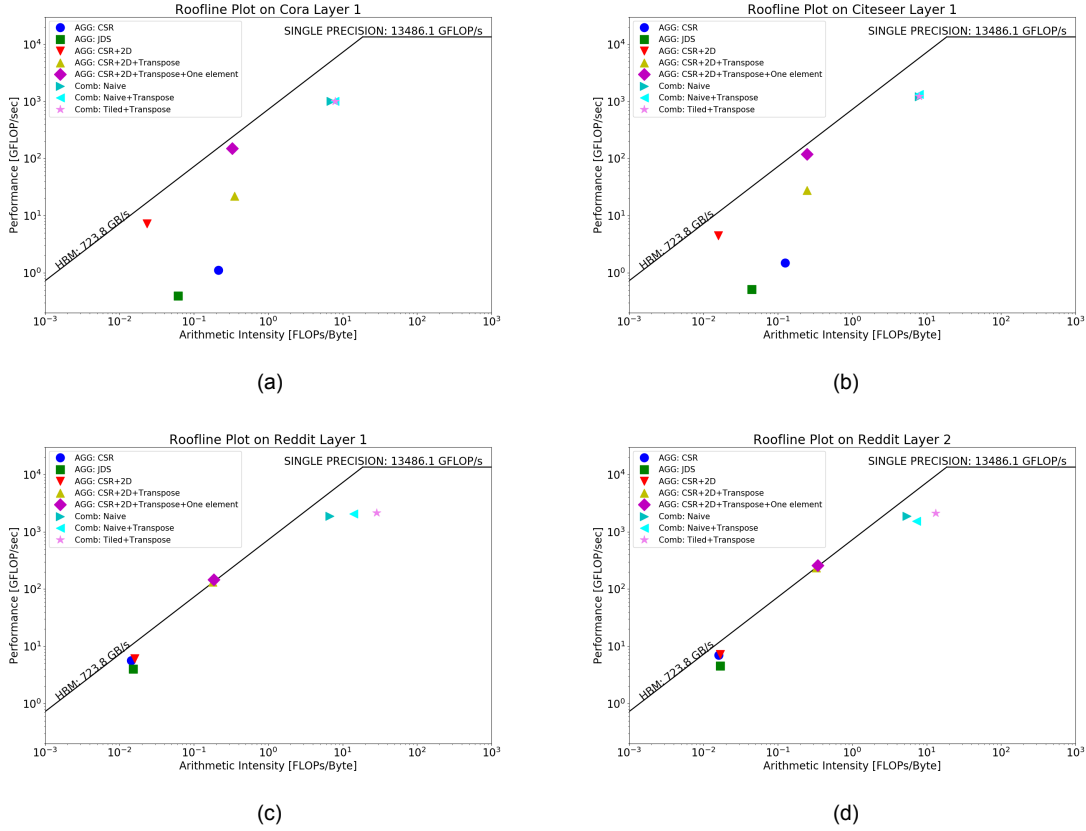


Figure 2. Roofline plots on different dataset and layers. Plots generated by scripts from [6]. These values are based on real measurements by NVIDIA's profiler instead of theoretical values. Arithmetic Intensity (AI) is calculated based on read/write operations on global HBM memory only. Read/write operations on L1/L2 cache are excluded. For cora and citeseer dataset, we only show these kernels on layer 1, since the dataset scale is small, data is cached in L1/L2 cache, there are almost 0 read/write operations on layer 2.³ From these roofline plots we can see our aggregation kernels are now limited by HBM bandwidth. And tiling significantly reduced DRAM access in the reddit dataset. There is still some room for improvement for aggregation and combination kernels. For the JDS kernel, we found it's leading to 7x-10x more DRAM write operations and that might come from the uncertainty of row order and causes longer running time.

Layout	Aggregation		Combination		Memory	Analyzer	Streams	Overall Runtime (5 runs average)		
Transposed	CSR	2D block + one element/thread	Naive	Tiled	Pinned	Reduction	Enabled	Cora	Citeseer	Reddit
								0.811s	2.923s	2281.393s
	✓		✓					0.2248s	0.6682s	44.2924s
✓	✓	✓		✓				0.2048s	0.6100s	37.4492s
✓	✓	✓		✓	✓			0.2016s	0.5950s	37.1758s
✓	✓	✓		✓	✓	✓		0.1930s	0.5878s	37.3066s
✓	✓	✓		✓	✓	✓	✓	0.1938s	0.5944s	37.3820s

Table 7. Overall run time of our optimized program. We show the overall run time of several configurations of our optimized programs. The first row shows the serial version's runtime on three datasets. We get around 4.2x speed-up in cora, 5.0x speed-up in citeseer and 61.2x speed-up in reddit dataset. The accelerated programs are now heavily limited by parsing cost (parsing features, parameters & graphs) on the host and can hardly be optimized by a GPU.⁴

³ Detailed measurements of FLOP, HBM usage can be found at <https://docs.google.com/spreadsheets/d/1O79LVxQoLMNhKS1xeU8-Ba4IGFOH53KIMIUSva72s9g/edit?usp=sharing>

⁴ Detailed total run time data for various combination of optimizations can be found at <https://docs.google.com/spreadsheets/d/1AvwBxucnRj3ybE6McdJTsikuXJ3klpxWfTRq1U1LtBc/edit?usp=sharing>

5. Conclusion

In this project, we implement the forward propagation of a Graph Convolutional Network (GCN) on a GPU. We optimize the two main steps in GCNs, aggregation and combination, as well as other components of the code like data transfer between host-device and analyzer function. From all the tested optimizations, we found that the best performance for GCN is achieved with the following enhancements clubbed together -

- Storing data set to pinned memory
- 2D Transposed CSR Aggregation
- Transposed Tiled Combination
- GPU Compare + Reduction Analyzer

The above setup helped us achieve a total speedup of 4.2x for cora dataset, 5x for citeseer dataset, and 61.2x for reddit dataset. Considering individual functions, we observed a maximum improvement of 6699x for the aggregation kernel used in the 2nd layer of the reddit dataset and a 10419x improvement in the combination kernel for the 1st layer of the reddit dataset. At this stage, the accelerated programs are heavily limited by parsing cost (parsing features, parameters & graphs) on the host and can hardly be optimized by a GPU device.

6. Contributions

Rui Guo implemented the baseline version and the optimizations for the aggregation kernel. Naman Saxena implemented the baseline version and the optimizations for the combination kernel and analyzer function. We worked together with other optimizations including transpose, data transfer between host and device, streams etc. We worked together analyzing performance of these optimizations and writing the report.

7. References

- [1] Abadal, S., Jain, A., Guirado, R., López-Alonso, J., & Alarcón, E. (2021). Computing graph neural networks: A survey from algorithms to accelerators. *ACM Computing Surveys (CSUR)*, 54(9), 1-38.
- [2] Kirk, D. B., Hwu, W.-m. W. (2010). *Programming Massively Parallel Processors: A Hands-on Approach*. Burlington, MA: Morgan Kaufmann Publishers. ISBN: 9780123814722 0123814723
- [3] Itu, L. M., Moldoveanu, F., Suci, C., & Postelnicu, A. (2012). GPU enhanced stream-based matrix multiplication. *Bulletin of the Transilvania University of Brasov. Engineering Sciences. Series I*, 5(2), 79.
- [4] NVIDIA, Vingelmann, P., & Fitzek, F. H. P. (2020). *CUDA, release: 10.2.89*. Retrieved from <https://developer.nvidia.com/cuda-toolkit>
- [5] Yang C. and Williams S. (2019). Performance Analysis of GPU-Accelerated Applications using the Roofline Model. *GTC 2019*. <https://developer.download.nvidia.com/video/gputechconf/gtc/2019/presentation/s9624-performance-analysis-of-gpu-accelerated-applications-using-the-roofline-model.pdf>
- [6] C. Yang, R. Gayatri, T. Kurth, P. Basu, Z. Ronaghi, A. Adetokunbo, B. Friesen, B. Cook, D. Doerfler, L. Oliker et al., An Empirical Roofline Methodology for Quantitatively Assessing Performance Portability, in 2018 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC). IEEE, 2018, pp. 14-23.

8. Appendix

8.1 Instruction to run optimized code

1) Clone git repository with the optimized GPU implementation

```
-> git clone https://github.com/rrrrrguo/eecs598\_GCN  
-> cd eeecs598_GCN  
-> git branch project_final
```

2) Build after loading relevant modules

```
-> module load cuda  
-> module load gcc  
-> module load cmake  
-> ./build
```

3) Test on any dataset

- For timing run:
-> sbatch run_test cora # citeseer, reddit
- For profiling run:
-> sbatch run_test cora _build